Introduction to Embedded Systems

Course Code: 244066

Lab 6

STM32 Analog and Digital Interface

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Instructions

- Complete every task in the lab practice session.
- For each task, include a 2–5 line description along with a screenshot of both your code and its output.
- Submit your lab report by email to Imran.Nadeem@outlook.com, using the subject line format as:
- **ES24-[lab number]-[registration number]**.
- Lab report need to be submitted before **11:59 pm**.
- Submissions received after the deadline will not be accepted.

Overview

Overview of Learning Outcomes

By completing these simulated labs, you will:

- Understand Analog-to-Digital and Digital-to-Analog Conversion
- Configure ADC and DAC in STM32
- Learn Key ADC Parameters
- Understand Practical Applications of ADC/DAC
- Simulate and Visualize Analog and Digital Data

Setup Notes

- Device choice: create the project for the **STM32F103RC** (high-density), **not** the F103C4 used in earlier labs. The F103C4 has only one **ADC** and no **DAC**, and the chapter's DAC examples need a device that has one. The RC has **three ADCs**, a 2-channel **DAC** (PA4/PA5), **DMA1**/**DMA2** and TIM1–8, plus 48 KB of RAM. Select CMSIS Core + Device Startup as before.
- Use #include "stm32f10x.h", <stdint.h> and <stdio.h>. Set the Debug tab to Use Simulator, reuse your STMaccess.ini (MAP 0x40000000,0x400FFFFF READ WRITE), set Xtal to 72 MHz, and use optimization -O0.
- Set the **ADC** clock to **12 MHz** (the limit is 14 MHz): RCC->CFGR &= ~RCC_CFGR_ADCPRE; RCC->CFGR |= RCC_CFGR_ADCPRE_DIV6;.
- Use the Watch window (Hex) for all checks: ADC1->..., DAC->..., DMA1_Channel1->..., and your own variables. If your build offers Peripherals → A/D Converter or D/A Converter windows, you may use them to set input voltages, but the Watch window is the main method.
- The simulator may not convert, calibrate or move data by itself. Force values instead:
  - If a **calibration** loop hangs, clear **CAL** or **RSTCAL** in ADC1->CR2.
  - If a wait on **EOC** hangs, set **ADC1-**>SR = 0x2 and force ADC1->DR.
  - Pend **interrupts** with NVIC->ISPR[0] (ADC1_2 is IRQ 18 → 0x00040000).
  - When a task says "**fallback**", model the hardware by hand and say so in your report.
- Treat the 12-bit conversion as mV = code × 3300 / 4095

Task 1: Single-Channel ADC1 Read (Polled) on PA1

Steps

1. Create the project, add main.c, and enable the clocks: RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_ADC1EN;. Set the ADC prescaler to divide by 6.
2. Configure PA1 as analog input by clearing its 4-bit field: GPIOA->CRL &= ~(0xF << 4);. Do not enable any pull resistor.
3. Select channel 1 as the only conversion: ADC1->SQR1 = 0; (length = 1) and ADC1->SQR3 = 1;.
4. Set the sampling time for channel 1 to 239.5 cycles: ADC1->SMPR2 = (7 << 3);.
5. Choose software triggering: ADC1->CR2 = ADC_CR2_EXTSEL | ADC_CR2_EXTTRIG;. Then power the ADC on: ADC1->CR2 |= ADC_CR2_ADON; and wait a short delay.
6. Calibrate: set RSTCAL and wait until it clears, then set CAL and wait until it clears.
7. Write uint16_t ADC_Read(void). It sets SWSTART, waits for EOC in ADC1->SR, and returns ADC1->DR.
8. In main(), call it in a loop and store raw and mv = (uint32_t)raw * 3300 / 4095.
9. Add ADC1->CR2, ADC1->SQR3, ADC1->SMPR2, ADC1->SR, ADC1->DR, raw and mv to the Watch window. Verify that SMPR2 reads 0x38, SQR3 reads 0x1, and CR2 reads 0x001E0001 before the first conversion.
10. Test the inputs 0, 1024, 2048, 3072 and 4095. Expected mV: 0, 825, 1650, 2475, 3300.
11. In 2 sentences, explain why PA1 must be analog with no pull-up or pull-down, and what one LSB is for this ADC (about 0.8 mV).

Task 2: LM35 Temperature Reading: Continuous Mode, Averaging and Sampling Time

Steps

1. Reuse the ADC setup from Task 1, but use PA2 (channel 2): GPIOA->CRL &= ~(0xF << 8); and ADC1->SQR3 = 2;.
2. Switch to continuous mode: add ADC_CR2_CONT to CR2, calibrate, and start with a single SWSTART. Reading DR clears EOC, so conversions keep repeating.
3. Write uint16_t ADC_Average8(void) that waits for EOC eight times, sums the eight readings and returns sum >> 3.
4. Convert with the LM35 rule of 10 mV per °C: temp_x10 = (uint32_t)avg * 3300 / 4095; (the mV value equals tenths of °C). Print or store temp_c = temp_x10 / 10 and temp_frac = temp_x10 % 10.
5. To test the averaging, temporarily replace the eight ADC reads with the array {308,312,310,311,309,313,310,311}. Verify avg = 310 and the result is 24.9 °C.
6. Force single values into DR: 311 should give 25.0 °C, and 1241 should give 100.0 °C. Fill a small table.
7. Try three sampling times for channel 2 (SMPR2 bits 8:6): 1.5 cycles (0x000), 55.5 cycles (0x140) and 239.5 cycles (0x1C0). Check the register value for each.
8. For each setting, calculate the conversion time (Tsamp + 12.5) / 12 MHz and the maximum rate: about 1.17 µs (857 kSPS), 5.67 µs (176 kSPS) and 21 µs (47.6 kSPS).
9. In 3 sentences, explain why a high-impedance sensor needs a longer sampling time, why a longer time is more accurate but slower, and why averaging reduces noise.

Task 3: Multi-Channel Scan Mode with DMA

Steps

1. Enable the clocks for GPIOA, ADC1 and DMA1 (RCC->AHBENR |= RCC_AHBENR_DMA1EN;). Set PA1, PA2 and PA3 to analog by clearing GPIOA->CRL bits 15:4.
2. Configure a three-channel sequence: ADC1->SQR1 = (2 << 20); (3 conversions) and ADC1->SQR3 = 1 | (2 << 5) | (3 << 10);. Verify that SQR3 reads 0x0C41.
3. Set 55.5-cycle sampling for all three channels: ADC1->SMPR2 = (5 << 3) | (5 << 6) | (5 << 9);. It should read 0x0B68.
4. Enable scan mode: ADC1->CR1 = ADC_CR1_SCAN;. Set CR2 with CONT, DMA, software trigger bits and ADON: ADC1->CR2 = ADC_CR2_CONT | ADC_CR2_DMA | ADC_CR2_EXTSEL | ADC_CR2_EXTTRIG | ADC_CR2_ADON; (reads 0x001E0103). Calibrate.
5. Declare volatile uint16_t adc_buf[3];. Configure DMA1 channel 1 as in Chapter 6: CPAR = (uint32_t)&ADC1->DR;, CMAR = (uint32_t)adc_buf;, CNDTR = 3;, CCR = 0x5A0; (memory increment, circular, 16-bit sizes, peripheral to memory). Then set EN (it should read 0x5A1).
6. Start with ADC1->CR2 |= ADC_CR2_SWSTART;.
7. Add adc_buf[0..2], DMA1_Channel1->CNDTR, ADC1->CR1, ADC1->CR2 and DMA1->ISR to the Watch window.
8. Verify that adc_buf[0], [1], [2] hold channels 1, 2 and 3 and that CNDTR counts 3 → 0 → 3 again because of circular mode. Fallback: write three different values into adc_buf and step CNDTR by hand, noting which SQR3 slot feeds each element.
9. In 3 sentences, explain why scan mode normally needs DMA (a single DR register is overwritten by the next channel), what circular mode does here, and why this frees the CPU.

Task 4: DAC Output Ramp on PA4 (Software Trigger)

Steps

1. Enable the clocks: RCC->APB1ENR |= RCC_APB1ENR_DACEN; and RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;.
2. Configure PA4 as analog: GPIOA->CRL &= ~(0xF << 16);.
3. Enable channel 1 with a software trigger: DAC->CR = DAC_CR_EN1 | DAC_CR_TEN1 | DAC_CR_TSEL1; (TSEL = 111). It should read 0x3D.
4. Write void DAC_Write(uint16_t v): DAC->DHR12R1 = v & 0x0FFF; DAC->SWTRIGR = 1;.
5. In main(), step v from 0 to 4095 in steps of 256 (and finish on 4095), calling DAC_Write(v), a short delay, and storing vout_mv = v * 3300 / 4095.
6. Add DAC->CR, DAC->DHR12R1, DAC->DOR1 and vout_mv to the Watch window. Verify that DOR1 follows DHR12R1 after each trigger.
7. Check these points: 0 → 0 mV, 1024 → 825 mV, 2048 → 1650 mV, 3072 → 2475 mV, 4095 → 3300 mV. Compare 2048 with the chapter's "about 1.65 V".
8. Set BOFF1 (bit 1) to disable the output buffer so CR reads 0x3F. Note the change in your report. The chapter says the buffer smooths and strengthens the output.
9. Fallback if the DAC is not modelled: copy DHR12R1 into DOR1 by hand after each trigger and say so in your report.
10. Make a table for 8-bit, 10-bit and 12-bit resolution: number of levels (256, 1024, 4096) and LSB size for a 3.3 V reference.

Task 5: ADC-to-DAC Loopback with Scaling and Clipping

Steps

1. Put the ADC code from Task 1 (PA1) and the DAC code from Task 5 (PA4) in one project. Initialise both.
2. Declare volatile uint8_t mode = 0; and write uint16_t process(uint16_t raw): mode 0 returns raw (pass-through), mode 1 returns raw / 2 (half the voltage), and mode 2 returns raw clipped to 3102 (about 2.5 V, since 2500 × 4095 / 3300 is about 3102).
3. In the main loop: raw = ADC_Read(); out = process(raw); DAC_Write(out);.
4. Add mode, raw, out, ADC1->DR, DAC->DHR12R1 and DAC->DOR1 to the Watch window. Change mode from the Watch window while running.
5. For each mode, force ADC1->DR to 1000, 2048, 3500 and 4095 and record out and the expected output voltage in mV.
6. Check these results: mode 0 outputs 1000, 2048, 3500, 4095; mode 1 outputs 500, 1024, 1750, 2047; mode 2 outputs 1000, 2048, 3102, 3102.
7. Explain why the output is off by up to 1 LSB in mode 1 (integer division) and why both converters being 12-bit makes mode 0 an exact copy.
8. In 2–3 sentences, relate your loop to the ADC → processor → DAC block diagram in the chapter, and give a real use of the clipping mode (for example, protecting an amplifier input).

STATUS: OPTIONAL

A Challenge Task (optional)

Optional practical extension for the embedded systems lab6.

REWARD

2× Bonus Marks

Applied directly to the current lab Score.

REQUIREMENT

Submit ≤ 09:00 PM

Complete alongside regular tasks during lab hours.

Challenge Task : Full Chain: Timer → ADC → DMA → USART Report

Steps

1. Reuse your USART1 initialisation from Chapter 5 (9600 baud, 8N1, PA9 TX). Add USART1_SendString().
2. Configure TIM3 for 100 Hz: PSC = 7199, ARR = 99, with CR2 = (2 << 4).
3. Configure ADC1 for channel 1 with the timer trigger and DMA: ADC1->CR2 = (4 << 17) | ADC_CR2_EXTTRIG | ADC_CR2_DMA | ADC_CR2_ADON; (reads 0x00180101). Calibrate.
4. Declare volatile uint16_t buf[16];. Configure DMA1 channel 1 as in Task 3 but with CNDTR = 16 and CCR = 0x5A1.
5. Start the timer. In the main loop, wait about 500 ms (using a SysTick millis() from Chapter 7 or a simple counter), and also increment idle_loops.
6. Every 500 ms, compute avg of the 16 buffer entries and mv = avg * 3300 / 4095. Format a message with sprintf(msg, "ADC=%4u mV=%4u\r\n", avg, mv); and send it with USART1_SendString(msg).
7. Calculate the message length (about 18 characters) and its time at 9600 baud with 10 bits per character (about 18–20 ms).
8. Test with all 16 buffer values set to 0, 2048 and 4095 (force them by hand if DMA is not modelled). Expected messages: ADC= 0 mV= 0, ADC=2048 mV=1650, ADC=4095 mV=3300.
9. Check the UART #1 window if available. Otherwise, inspect msg in the Watch window.
10. Draw the chain (TIM3 → ADC1 → DMA1 → memory → CPU → USART1 → PC), label what each block does, and state in 3 sentences how CPU load is reduced compared with polling the ADC and sending each reading yourself.